Prolog — Reference

Prolog here is an interpreter (written in our C, compiled to .NET IL by `cc`): a precedence-climbing term reader plus a real engine — **unification**, a binding **trail**, SLD resolution with **backtracking**, and **cut**. You state facts and rules; you ask queries; the engine searches for all answers. (Prolog is declarative, so the tutorial activities below are adapted to that model.)

```
prolog prog.pl        # load clauses; run :- / ?- directives, printing solutions
```

Quick Reference

```
fact(tom, bob).                       % a fact
grand(X, Z) :- parent(X, Y), parent(Y, Z).   % a rule (:- = "if", , = "and")
?- grand(tom, Who).                   % a query — prints each solution's bindings
X = 5      Atom = lower-case-or-'quoted'      Var = Capitalized-or-_
[H | T]   [a, b, c]   []              % lists (H = head, T = tail)
=  \=  ==     is   < > =< ≥ =:= =\=  % unify / not-unify / equal ; arithmetic + compare
, ;  →  \+  !   call                  % and / or / if-then / not / cut
write  writeln  nl                    % output
```

Data types

Prolog terms: **atoms** (`tom`, `+`, `'with space'`), **numbers**, **variables** (start with an uppercase letter or `_`), **compound terms** (`parent(tom, bob)`, functor + arguments), and **lists** (`[a, b | T]`, sugar for the `.`/2 functor ending in `[]`). There is no separate string type — text is atoms.

Statements / Commands

A program is a sequence of **clauses** ended by `.`: facts (`p(a).`), rules (`head :- body.`), and **directives** (`:- Goal.` or `?- Goal.`) that run immediately. A body combines goals with `,` (and), `;` (or), `→` (if-then), `\+` (negation as failure), and `!` (cut). Built-ins: `=`, `\=`, `==`, `is`, the comparisons, `write`/`writeln`/`nl`, `call`.

Functions

Prolog has no functions — it has **predicates** (relations). A predicate succeeds or fails and may bind variables; the same predicate can run "both directions" (e.g. `append/3` both joins and splits lists). Arithmetic *functions* live on the right of `is` (`X is Y + 1`).

Input / Output

`write(T)` prints a term; `writeln(T)` adds a newline; `nl` prints a line break. A directive query prints, for each solution, the bindings of its named variables, or `false.` if there are none.

Graphics

None. Activity 8 below is **text art** built from `write` / `nl` in a recursive predicate.

Notes

- The engine: unification with an occurs-check-free binding trail; depth-first search with chronological backtracking; `!` (cut) prunes choice points.
- A prelude (in Prolog) defines `append`, `member`, `length`, `reverse`, `last`, `between`.
- Directives enumerate **all** solutions (capped at 100) and print each variable binding.

Subset boundaries

A solid teaching Prolog, not full ISO. Not included: `assert` / `retract` (dynamic database), `findall` / `bagof` / `setof`, `op/3` (user operators), real strings/atoms distinction, DCGs, exceptions, and full I/O. If-then-else `(C → T ; E)` proves `C` once (fine for ground guards).

Tutorial

Every example was run with `prolog`; the output shown is real.

1. Your first program

```
likes(sam, pizza).
likes(sam, sushi).
likes(deb, sushi).
?- likes(sam, What).
```

```
?- likes(sam,_What).
What = pizza
What = sushi
```

Two facts about `sam`; the query asks "what does sam like?" and backtracking yields **both** answers.

2. Variables and data types

Terms are atoms (`pizza`), numbers (`5`), variables (`What` — capitalized), compounds (`likes(sam, pizza)`), and lists (`[a, b, c]`). A variable is a hole that unification fills; the same query above bound `What` to each matching atom in turn.

3. Flow control

"Control" is search: conjunction, backtracking, and **cut**.

```
max(X, Y, X) :- X ≥ Y, !.
max(_, Y, Y).
?- max(3, 7, M).
?- max(9, 2, M).
```

```
?- max(3,7,_M).
M = 7

?- max(9,2,_M).
M = 9
```

The `!` (cut) commits to the first clause once its guard holds, so `max` gives exactly one answer instead of backtracking into the fallback.

4. Arrays

Prolog's structure is the **list**; the prelude provides the relational toolkit:

```
?- append(A, B, [1,2,3]).
?- member(M, [a,b,c]).
?- reverse([1,2,3,4], R).
?- length([a,b,c,d], N).
```

```
A = [], B = [1,2,3]
A = [1], B = [2,3]
A = [1,2], B = [3]
A = [1,2,3], B = []
M = a
M = b
M = c
R = [4,3,2,1]
N = 4
```

`append/3` is relational — given the whole list it enumerates **every** way to split it. `member` backtracks through elements.

5. Subroutines and functions

The unit of reuse is the **rule**. Rules chain goals and recurse:

```
parent(tom, bob).
parent(bob, ann).
parent(bob, pat).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
fact(0, 1).
fact(N, F) :- N > 0, N1 is N - 1, fact(N1, F1), F is N * F1.
?- grandparent(tom, Who).
?- fact(6, F).
```

```
Who = ann
Who = pat
F = 720
```

`grandparent` composes two `parent` goals; `fact` recurses, using `is` for arithmetic.

6. Memory management

Prolog has no manual memory — the engine manages the **logic variable** machinery:

- **Unification** binds variables to terms; bindings are recorded on a **trail**.
- On **backtracking** the trail is unwound, undoing bindings so an alternative can be tried — this is what makes `append(A,B,[1,2,3])` produce every split.
- Each clause use **renames** the clause's variables to fresh ones, so recursion doesn't clash. `between` shows the generator pattern:

```
?- between(1, 5, X).
```

```
X = 1
X = 2
X = 3
X = 4
X = 5
```

7. Strings and a text layout

Prolog text is **atoms**, printed with `write` / `nl`. A rule can lay out a relation's solutions:

```
likes(sam, pizza).
likes(sam, sushi).
likes(deb, sushi).
agree(X, Y, F) :- likes(X, F), likes(Y, F).
?- agree(sam, deb, Food).
```

```
Food = sushi
```

`agree` finds food two people both like by unifying the shared variable `F` across two goals — the "layout" is the set of bindings the engine reports.

8. Drawing a picture

No graphics — **text art** from a recursive predicate using `write` / `nl`:

```
stars(0) :- !.
stars(N) :- write(*), N1 is N - 1, stars(N1).
tri(Max, Max).
tri(Max, I) :- I < Max, stars(I), nl, I1 is I + 1, tri(Max, I1).
?- tri(5, 1).
```

```
*
**
***
****
true
```

`stars(N)` writes `N` asterisks; `tri` recurses over the rows. (`true` is the query succeeding.)

9. Where to go next

Lean into the relational view: write predicates that work in multiple directions (like `append`), use cut to make deterministic ones, and let backtracking enumerate. Next steps are the bigger built-ins — `findall`, `assert` / `retract`, user operators (`op/3`) — listed under *Subset boundaries*.